

System Architecture

PG-S2-47 — Hydrogen Production Materials Discovery Using Deep Neural Networks

2 October 2026

Project: PG-S2-47 — Hydrogen Production Materials Discovery Using Deep Neural Networks, Adelaide University College of Engineering & IT **Scope:** this document is the expanded, explained version of the architecture frozen in Sprint 1 and specified in `glossary.md` §2–3. It describes what each part of the system is, why it is shaped the way it is, and how the pieces fit together, as of 2 October 2026.

1. Why the architecture is shaped this way

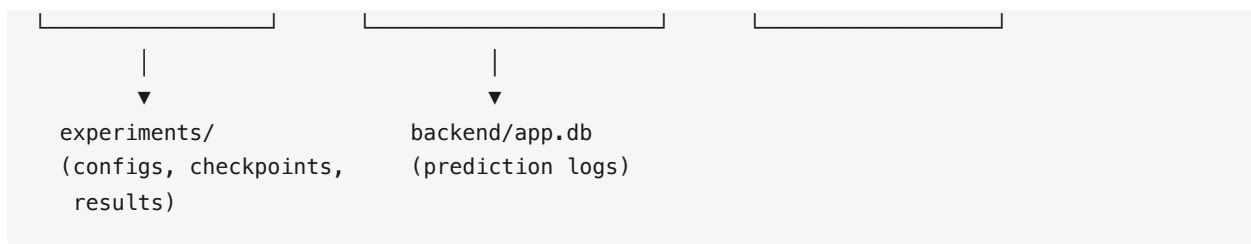
The project’s actual technical deliverable is a research *methodology*: a graph neural network that predicts a molecule’s energy and interatomic forces, together with a trustworthy estimate of when that prediction should not be trusted. That is proved out on MD17 — a standard molecular dynamics benchmark — before it would ever be pointed at real hydrogen-production catalyst screening.

Two consequences follow directly from that goal, and both are visible throughout the architecture:

1. **The research code must be usable without a web application around it.** A thesis-grade result has to be reproducible from the command line, scorable by an automated test, and comparable across five different model architectures trained by five different people. The system is therefore built so the modelling layer (`ml/`) never depends on the API or the dashboard — it is a standalone, independently testable package, and the backend is a consumer of it, not the other way round.
 2. **Every model the team builds must be interchangeable.** Five architectures (MPNN, SchNet, PaiNN, DimeNet++, PhysNet) are being compared head-to-head as Phase 1 candidates, and two more uncertainty-aware variants (BLIP, graph-space stochasticity) are built on top of whichever one wins. None of that is possible unless every model exposes the exact same interface and reads the exact same data. That shared contract — not any one model’s code — is the single most load-bearing design decision in the system, and it is described in detail in §3.3 below.
-

2. The three layers





The dependency arrow only ever points one way: `ml` → `backend` → `frontend`. The reverse is never true — the modelling code has no knowledge that an API or a dashboard exists, and the API has no knowledge of the dashboard beyond the HTTP contract it exposes. This means each layer can be built, run, and tested in isolation, and it means the research work (the part that is actually assessed as the project’s technical contribution) is never put at risk by a change to the web application around it.

Layer	What it is	What it depends on
<code>ml/</code>	The research core — data pipeline, model implementations, training and evaluation code	Nothing else in the repository
<code>backend/</code>	A FastAPI service that loads a trained checkpoint from <code>ml/</code> and serves predictions over HTTP	<code>ml/</code> only
<code>frontend/</code>	A React/Vite dashboard	<code>backend/</code> ’s HTTP API only

3. The `ml/` layer, in detail

This is the layer the project is actually judged on, and it is organised into four sub-packages plus one shared configuration file.

3.1 `ml/data/` — the data pipeline

Owner: Shijin Mathew.

File	Responsibility
<code>md17.py</code>	Loads raw MD17/MD22 trajectory files (<code>z</code> , <code>R</code> , <code>E</code> , <code>F</code> — atomic numbers, positions, energy, forces) from both source formats found in the raw data (plain <code>.npz</code> , and <code>.zip</code> archives containing pre-split train/test members as either nested <code>.npz</code> or extended-XYZ text). Also builds the neighbour graph for one molecular configuration.
<code>preprocessing.py</code>	Promotes data through the bronze → silver → gold pipeline described in §4 below. Runnable directly as

File	Responsibility
	<code>python -m ml.data.preprocessing .</code>
<code>datasets.py</code>	A PyTorch Geometric <code>Dataset</code> wrapper that reads <i>only</i> from the gold tier and produces one <code>Data(x, pos, edge_index, edge_attr, y, force)</code> object per molecular configuration — the exact shape every model in <code>ml/models/</code> is built to consume.

3.2 `ml/models/` — one file per architecture

Owner: Ruturaj Yashwant Bhosale (lead), with every team member contributing one Phase 1 entry.

Every model in this folder — regardless of architecture — implements exactly two methods:

```
predict_energy(z, edge_index, edge_attr, batch=None) -> energy
predict_energy_and_forces(z, pos, edge_index, edge_attr, batch=None) -> (energy, forces)
```

`predict_energy` is used where only an energy prediction is needed (e.g. quick evaluation).

`predict_energy_and_forces` additionally differentiates the predicted energy with respect to atomic position ($\text{forces} = -\partial E / \partial \mathbf{R}$, via autograd) — the physically correct way to obtain forces from an energy model, and the thing every one of the five bake-off implementations had to get right independently (see the shared contract note in §3.3).

File	Architecture	Status as of this report
<code>mpnn.py</code>	Gilmer-style message passing with RBF-expanded distance edges — the project's original Phase 1 default	Implemented; shared training loop built alongside it
<code>schnet.py</code>	Continuous-filter convolutions (Schütt et al. 2017)	Implemented and fully trained — the only fully comparable Phase 1 result as of this report
<code>painn.py</code>	Polarizable Atom Interaction NN (Schütt, Unke & Gastegger 2021) — equivariant, carries directional (not just distance) information between atoms	Implemented and tested; training pending
<code>dimenet.py</code>	Directional Message Passing NN++ (Gasteiger et al.) — uses angular as well as radial information	Implemented and trained, on a different data subset than the other four (see §6)
<code>physnet.py</code>	Atom-wise additive energy contributions (Unke & Meuwly	Implemented, tested, and trained on a reduced-size subset due to

File	Architecture	Status as of this report
	2019), short-range term only	CPU training-time constraints
<code>registry.py</code>	A model-name → class lookup, with a standardised checkpoint format recording which architecture, configuration, and dataset produced it	Added during the bake-off once the team recognised the evaluation code could not otherwise stay model-agnostic
<code>blip.py</code>	Phase 2 — BLIP-style input-dependent Gaussian stochasticity in the model's weights	Not yet implemented
<code>graph_stochastic.py</code>	Phase 3 — the team's own contribution: stochasticity moved into node/edge representations instead of weights	Not yet implemented

3.3 The shared contract — why this is the architecture's load-bearing joint

Two things make the five Phase 1 models comparable at all, and both were hard-won during Sprint 3:

- **Identical input shape.** `ml/data/datasets.py` already expands interatomic distance into a 16-dimensional Gaussian radial basis before any model sees it. A model that is only given a raw scalar distance, or that re-expands an already-expanded feature, is silently seeing different information from the others — this was caught and corrected independently in three of the five implementations.
- **A genuine gradient path from positions to forces.** Because forces are obtained by differentiating energy with respect to position, a model that computes its edge features once, outside the differentiated computation, produces forces that are exact zeros — with no error raised anywhere. Every team member who built a Phase 1 entry found a version of this bug in their own or another branch's code during the sprint. It is now documented prominently in the model cards for MPNN, SchNet, and PhysNet specifically so it is not reintroduced in Phase 2 or Phase 3.

3.4 `ml/training/` — training and evaluation

Owner: Raturaj Yashwant Bhosale (training loop), Fazin Faizal (evaluation harness).

File	Responsibility
<code>train.py</code>	The shared, config-driven training loop. Dispatches on the architecture named in the experiment config and trains any registered model the same way, with the same combined energy+force loss.
<code>evaluate.py</code>	Loads a trained checkpoint via <code>ml/models/registry.py</code> and scores it on the

File	Responsibility
	held-out test split, without needing to know in advance which architecture it is scoring.
<code>benchmark.py</code>	Runs every trained Phase 1 (and later Phase 2/3) checkpoint through <code>evaluate.py</code> and assembles the single comparison table the <code>/benchmarks</code> API endpoint and the technical report both read from.
<code>train_schnet.py</code> , <code>train_dimenet.py</code> , <code>train_physnet.py</code>	Standalone, architecture-specific training scripts, used where the shared <code>train.py</code> was not yet available or not yet compatible with a given entry. Each is explicitly documented as a temporary stand-in to be retired once every entry runs through the shared loop.

3.5 `ml/utils/` — shared metrics and the data/metric contract

Owner: Fazin Faizal.

File	Responsibility
<code>metrics.py</code>	Streaming energy/force accuracy metrics (mean absolute error and root-mean-square error, in both total and per-atom conventions), accumulated correctly across mini-batches rather than naively averaged batch-by-batch — a common source of silently wrong benchmark numbers that this module is specifically built to avoid.
<code>contract.py</code>	A typed interface (<code>ModelOutput</code> , <code>ReferenceData</code> , <code>DatasetKey</code> , <code>Units</code>) between the data layer and the metric layer, so that unit mismatches and attempts to pool incomparable data (e.g. the same molecule at two different levels of quantum-chemical theory) are rejected structurally rather than relying on every caller remembering not to make that mistake.
<code>logging.py</code>	Shared run logging.

3.6 `ml/config.py` — the experiment configuration schema

Every training run is defined by a YAML file in `experiments/configs/`, never by hard-coded values in a script. The schema has three sections — `data` (which molecule, theory level, split, cutoff radius, random seed), `model` (architecture and its hyperparameters), and `train` (epochs, batch size, learning rate, the energy/force loss weighting) — matching the `ExperimentConfig` dataclass in this file exactly.

4. Data staging: bronze, silver, gold

Owner: Shijin Mathew. This is the discipline that makes every later model's results trustworthy, and it is deliberately simple:

Tier	Path	What happens here
Bronze	<code>data/bronze/md17/</code> , <code>md22/</code>	Raw, exactly as downloaded. Never edited in place. Tracked via Git LFS.
Silver	<code>data/silver/...</code>	Deduplicated (several source files in the raw download turned out to be redundant across formats), format-unified into one consistent <code>.npz</code> shape, and validated — shape, missing-value, and unit checks all run here.
Gold	<code>data/gold/...</code>	Production-ready. This is where the train/validation/test split is computed and saved <i>alongside</i> the data itself, as index arrays baked into the same file. This is the only tier <code>ml/data/datasets.py</code> is permitted to read.

Why the split lives inside the gold file rather than in code: molecular dynamics trajectories change very little from one timestep to the next, so a *random* split would place near-duplicate frames on both sides of the train/test boundary and produce an inflated, meaningless accuracy figure. The split used instead is **contiguous trajectory blocks** — the first 80% of simulated time for training, the next 10% for validation, and the final 10%, chronologically after everything the model has seen, for test. Because this split is computed once, deterministically, with no randomness involved, and saved into the gold file itself, every team member who regenerates gold data from the same bronze source gets byte-identical train/validation/test assignments — which is what makes the five-model Phase 1 comparison valid in the first place.

The graph representation built from this data, used identically by every model: each atom is a node (embedded by atomic number); an edge connects every pair of atoms within a 5 Ångström cutoff radius; and the edge feature is the interatomic distance, expanded into a 16-dimensional Gaussian radial basis. This was chosen over string-based representations such as SMILES specifically because it captures the three-dimensional geometry that an energy/force model needs and a 2D connectivity string does not.

5. The backend/ layer

Owner: Alex Tanui. A FastAPI service whose only job is to load a trained checkpoint from `ml/` and expose it over HTTP — it contains no modelling logic of its own.

File	Responsibility
<code>app/main.py</code>	Application entry point; registers the routes below.
<code>app/api/predictions.py</code>	<code>POST /predictions</code> — given a molecule and configuration, returns predicted energy and forces (and, once Phase 2/3 land, an uncertainty estimate).
<code>app/api/benchmarks.py</code>	<code>GET /benchmarks</code> — serves the comparison table produced by <code>ml/training/benchmark.py</code> .
<code>app/api/molecules.py</code>	<code>GET /molecules</code> — lists which MD17/MD22 molecules are available to predict against.
<code>app/services/inference.py</code>	The only place in the codebase that loads an <code>ml/</code> checkpoint and calls a model. Route handlers never touch model code directly.
<code>app/models/db.py</code>	SQLAlchemy tables logging every prediction request (SQLite for local development).
<code>app/models/schemas.py</code>	Pydantic request/response models, matching the API contracts in <code>glossary.md</code> §4 exactly.
<code>app/core/config.py</code>	Settings — database URL, checkpoint path, CORS.

6. The frontend/ layer

Owner: Alex Tanui. A React/Vite dashboard with three pages:

Page	Purpose
<code>Dashboard.tsx</code>	Landing page — project purpose and the three-phase research plan.
<code>Predict.tsx</code>	Pick a molecule and configuration, call <code>POST /predictions</code> , display the predicted energy/force/uncertainty.
<code>Benchmarks.tsx</code>	Chart/table comparing every trained model on <code>GET /benchmarks</code> .

Every page calls the backend exclusively through `src/api/client.ts` — no component is permitted to call `fetch()` directly, so there is exactly one place in the frontend that knows the shape of the backend's API.

7. experiments/ — where a training run's artefacts live

Path	Contents
experiments/configs/*.yaml	One file per run, matching the ml/config.py schema.
experiments/checkpoints/	Trained model weights. Deliberately excluded from version control — regenerated by re-running training, never committed.
experiments/results/	Per-run metrics and the combined benchmark table, produced by ml/training/evaluate.py / benchmark.py . Also excluded from version control, for the same reason.

8. Current implementation status

This architecture is fully specified, but not every box in the diagrams above is built yet, and the team's work is currently spread across five separate Git branches rather than consolidated into one. As of this report:

- **Fully implemented and merged to the shared history:** the data pipeline (ml/data/), the metrics/contract layer (ml/utils/), and the SchNet Phase 1 entry with a complete trained result.
- **Implemented, on individual branches, not yet merged:** the MPNN backbone and shared training loop; the PaiNN entry, the shared evaluation harness, and the model registry; the DimeNet++ entry; the PhysNet entry.
- **Not yet started:** Phase 2 (blip.py) and Phase 3 (graph_stochastic.py); wiring real (non-stub) predictions and uncertainty into the backend and frontend.

Consolidating the five branches and bringing every Phase 1 entry onto identical training conditions is the team's immediate priority before the Phase 1 baseline can be formally selected — see docs/sprint_reports.md for the detailed sprint-by-sprint account of how this status was reached.